
Inscription multi-tenant & multi-org

dans la plateforme KSM-ERP

Architecture du flux d'inscription et cycle d'onboarding collaborateur

Ce rapport décrit l'architecture et le flux d'inscription publique d'un collaborateur dans l'ERP en environnement multi-tenant, multi-org. Il détaille la résolution du tenant à l'aide d'un code d'organisation, la création sécurisée du compte via le BFF (proxy), et la politique d'affectation de rôles.

Auteur : ALD • azangueleone19@gmail.com

Composants impliqués	FRONTEND • BACKEND (BFF) • KSM_Kernel_Layer
Fichiers clés	login-modal.tsx • KernelProxyController.java • AuthController.java (Ke
Sécurité	Rôle par défaut (AIDE_COMPTABLE) • Cible résolue par code d'organisation
Date	8 juillet 2026

Table des matières

1	Vue d'ensemble et problématiques	2
2	Le flux d'inscription en deux étapes	2
2.1	Étape 1 — Résolution du Tenant (<code>discover-sign-up-contexts</code>)	2
2.2	Étape 2 — Création du compte (<code>sign-up</code>)	3
2.3	Absence de gestion de l'unicité globale des e-mails (Bug Kernel 500)	3
3	Le rôle du BFF (Backend-For-Frontend)	5
4	Cycle post-inscription	5
5	Synthèse	5

1. Vue d'ensemble et problématiques

Dans un système multi-tenant et multi-org tel que KSM-ERP, l'inscription d'un nouvel utilisateur à partir de l'écran d'accueil public pose deux défis majeurs :

1. **Identification du Tenant/Organisation** : Un utilisateur externe (anonyme) ne possède pas de jeton ni d'identifiant de tenant configuré en dur dans son navigateur. Il doit donc fournir une clé d'identification, sous forme de **Code d'organisation** (par exemple `KSM-CPTA-LA`), pour que le système puisse le diriger vers l'espace de travail correspondant.
2. **Sécurité des droits d'accès (Rôles)** : Permettre à un utilisateur externe de s'auto-attribuer un rôle d'administration (`ADMIN`, `RESPONSABLE_COMPTABLE`) lors de son inscription publique constitue une faille de sécurité critique. L'inscription doit assigner un rôle par défaut non-critique (par exemple `AIDE_COMPTABLE` ou collaborateur simple), les droits avancés étant gérés a posteriori par son responsable.

Règle d'or de l'inscription

Le compte utilisateur est créé dans le Kernel en tant que compte d'attente rattaché à l'organisation cible, avec des droits initiaux restreints. Seul un administrateur ou responsable connecté à cette organisation peut ensuite promouvoir son rôle via l'interface d'administration.

2. Le flux d'inscription en deux étapes

L'inscription publique se fait via le protocole d'authentification du Kernel en interrogeant deux endpoints à travers le BFF.

2.1 Étape 1 — Résolution du Tenant (`discover-sign-up-contexts`)

Le frontend soumet le code d'organisation saisi par l'utilisateur au Kernel afin de récupérer les métadonnées de l'organisation et le jeton de transaction d'inscription (`selectionToken`) :

```
POST {BFF}/api/kernel/auth/discover-sign-up-contexts
Content-Type: application/json

{ "organizationCode": "KSM-CPTA-LA" }
```

- **Si le code est incorrect** : Le Kernel renvoie un code `404 Not Found`, et le frontend affiche : *"Code d'organisation invalide ou introuvable"*.
- **Si le code est valide** : Le Kernel renvoie la structure suivante :

```
{
  "selectionToken": "eyJhbGciOi...",
  "expiresInSeconds": 300,
  "contexts": [
    {
      "contextId": "ctx-123",
      "tenantId": "11111111-2222-3333-4444-555555555555",
      "organizationId": "99999999-8888-7777-6666-555555555555",
      "organizationName": "KSM Comptabilite LA",
      "organizationCode": "KSM-CPTA-LA"
    }
  ]
}
```

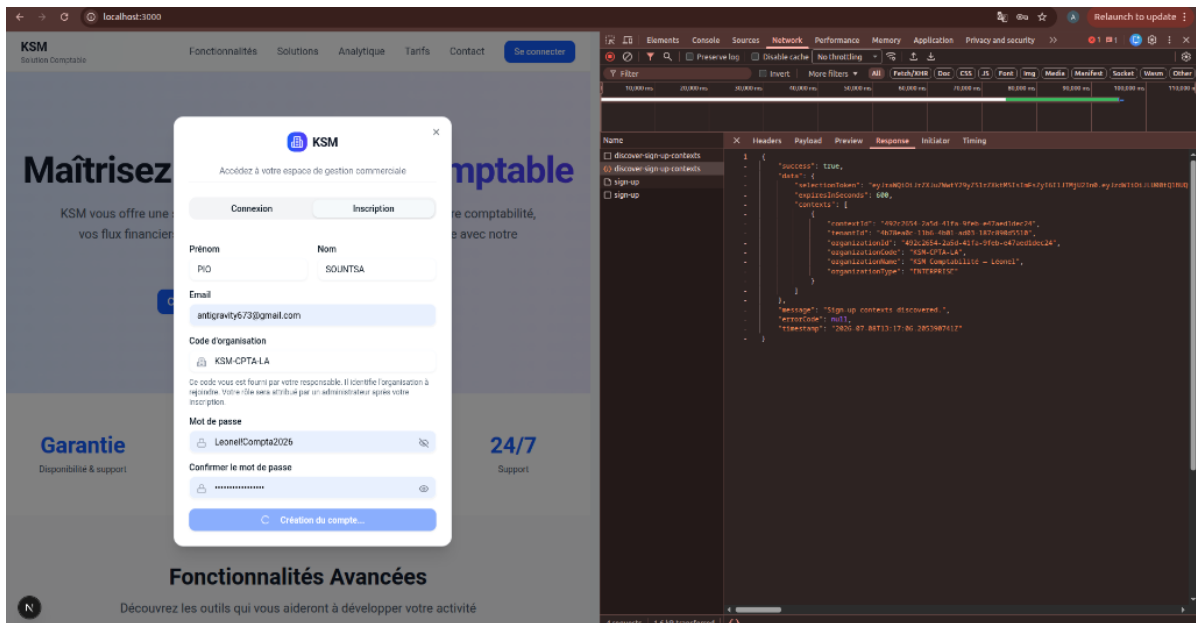


FIGURE 1 – Capture réelle — Formulaire d'inscription et réponse `discover-sign-up-contexts` (200 OK). On distingue le `selectionToken` et le contexte `KSM-CPTA-LA` dans l'onglet *Network* des DevTools.

2.2 Étape 2 — Création du compte (sign-up)

Le frontend extrait le `tenantId`, le `contextId` et le `selectionToken` reçus à l'étape 1, et soumet les informations personnelles pour procéder à l'inscription :

```
POST {BFF}/api/kernel/auth/sign-up
Content-Type: application/json

{
  "tenantId": "11111111-2222-3333-4444-555555555555",
  "signUpSelectionToken": "eyJhbGciOi...",
  "contextId": "ctx-123",
  "firstName": "Leonel",
  "lastName": "Azangue",
  "username": "leonel.azangue@ksm.dev",
  "email": "leonel.azangue@ksm.dev",
  "password": "*****",
  "socialProvider": "LOCAL",
  "accountType": "PROSPECT",
  "businessType": "INDIVIDUAL"
}
```

Valeurs autorisées pour `accountType` : le Kernel n'accepte que `PROSPECT` ou `BUSINESS`. La valeur `EMPLOYEE` est rejetée avec une erreur `400 AUTH_INVALID_REQUEST`. Lors d'une inscription publique, on utilise `PROSPECT` ; le responsable pourra ensuite promouvoir le compte.

Le Kernel vérifie le `signUpSelectionToken`, valide l'unicité de l'adresse email et procède à la création du compte dans le tenant désigné. En cas de succès, la réponse `201 Created` contient un champ `status` qui peut valoir `EMAIL_VERIFICATION_REQUIRED`, indiquant que l'utilisateur doit confirmer son adresse avant de se connecter.

2.3 Absence de gestion de l'unicité globale des e-mails (Bug Kernel 500)

Lors de nos tests, nous avons constaté qu'une tentative d'inscription avec une adresse e-mail déjà enregistrée produit également une erreur `500 Internal Server Error` au lieu d'une erreur

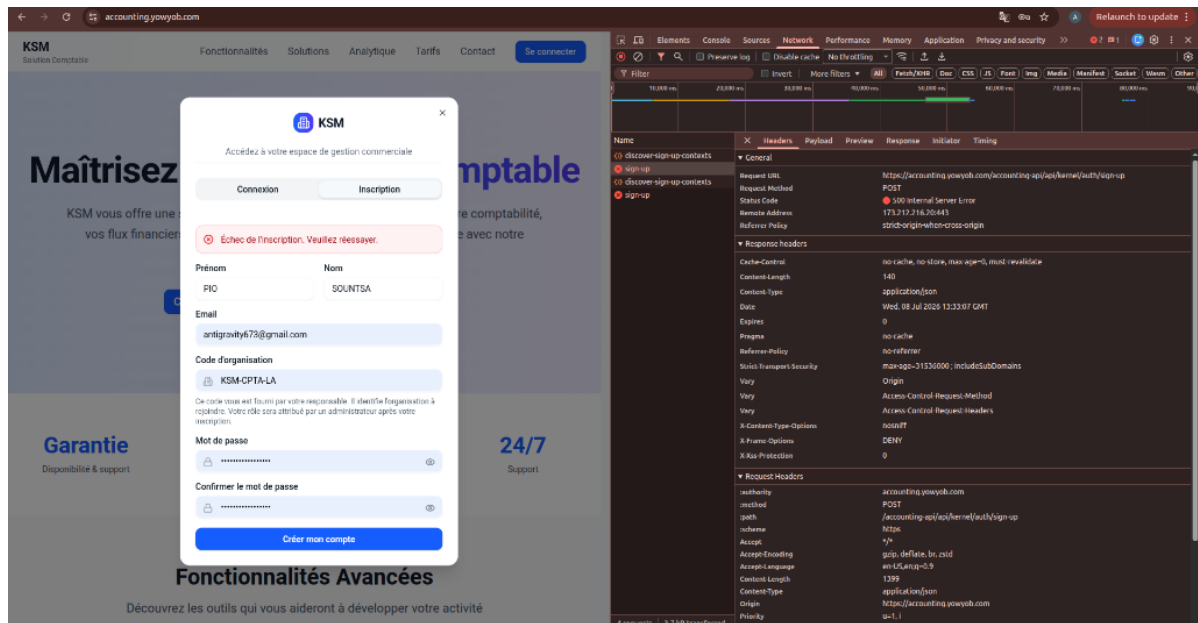


FIGURE 2 – Capture réelle — Erreur 500 Internal Server Error sur `/sign-up` causée par l'utilisation de `accountType: "EMPLOYEE"` (avant correction).

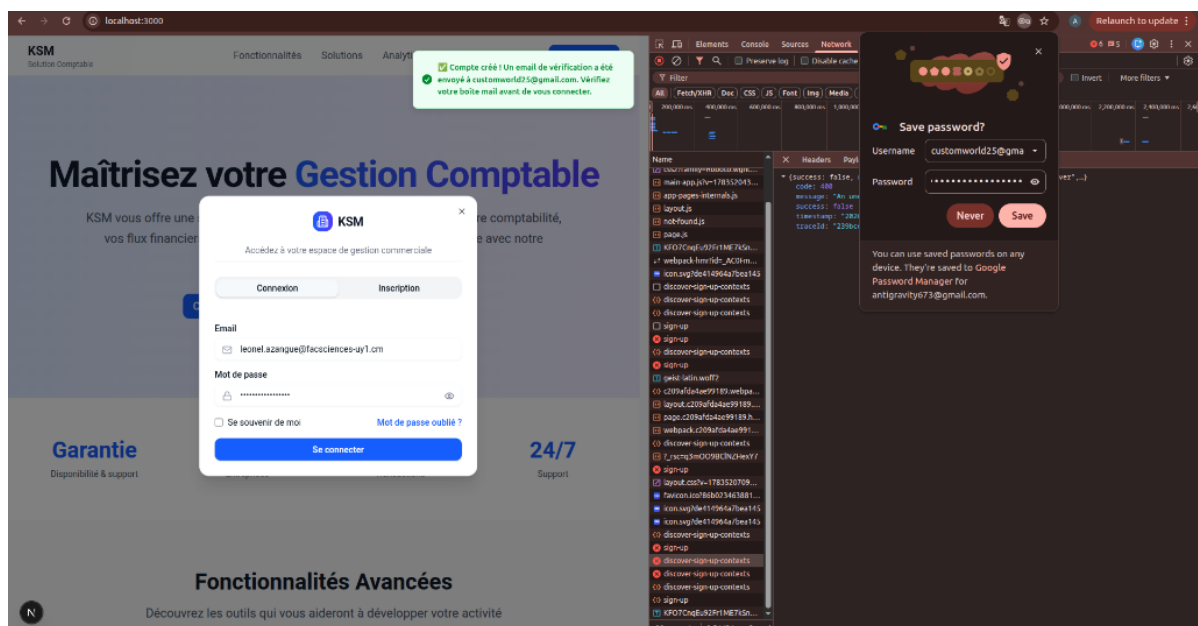


FIGURE 3 – Capture réelle — Message de succès affiché à l'utilisateur (201 Created) après correction du type de compte en `PROSPECT` et utilisation d'une adresse e-mail unique.

409 Conflict. L'analyse du code source du Kernel révèle la cause exacte de ce dysfonctionnement :

- **Vérification locale insuffisante** : Le service `AuthApplicationService.signUp` vérifie la disponibilité de l'adresse e-mail uniquement dans le tenant cible via `userAccountRepository.findByPrincipal(tenantId, email)`.
- **Levée d'exception par l'acteur** : Si l'e-mail est déjà utilisé dans une autre entité active, l'appel subséquent à `createActorUseCase.createActor` lève une exception de domaine `DuplicateActorEmailException`.
- **Absence de gestionnaire dans le flux d'inscription** : Le point de terminaison `/sign-up` est contrôlé par `AuthController`, qui utilise le gestionnaire d'exceptions `AuthExceptionHandler`. Ce dernier ne gère pas `DuplicateActorEmailException` (celle-ci n'est déclarée que dans `ActorExceptionHandler`,

dédié au contrôleur d'acteurs).

- **Effet** : Non capturée, l'exception remonte sous la forme d'une erreur technique globale avec un statut HTTP 500, induisant le frontend en erreur.

3. Le rôle du BFF (Backend-For-Frontend)

Le frontend ne communique jamais directement avec le Kernel de production pour des raisons évidentes de sécurité. Les secrets de signature d'application (`X-Client-Id` et `X-API-Key`) ne doivent pas être exposés côté client.

Le composant `BACKEND` agit comme un reverse-proxy transparent grâce à son contrôleur `KernelProxyController`.

- Tout appel vers `/api/kernel/**` est capturé par le BFF.
- Le BFF y injecte automatiquement les en-têtes d'identification requis par le Kernel (`X-Client-Id` et `X-API-Key`).
- Il transmet la requête vers l'URL du Kernel (`auth.api.url`) et relaie la réponse au client.

Ainsi, le flux d'inscription s'exécute de façon parfaitement sécurisée, sans exposer les clés d'infrastructure de la plateforme KSM.

4. Cycle post-inscription

Dès que la création du compte est confirmée par le Kernel (statut `201 Created`), le frontend inspecte le champ `data.status` de la réponse :

1. **Vérification email requise** (`EMAIL_VERIFICATION_REQUIRED`) : le Kernel a envoyé un e-mail de confirmation. Le frontend redirige vers l'onglet de connexion et affiche un message invitant l'utilisateur à vérifier sa boîte mail. La connexion n'est pas tentée, car le Kernel la refuserait tant que l'email n'est pas vérifié.
2. **Compte immédiatement actif** : le frontend enchaîne automatiquement la découverte des contextes (`POST /api/auth/discover-contexts`), la sélection du contexte d'organisation, et la redirection vers `/accounting/dashboard`.

Note : en mode strict, le Kernel rejette toute tentative de connexion (`login`) pour un compte `LOCAL` dont l'email n'est pas encore vérifié (exception `EmailNotVerifiedException`). La gestion de ce statut côté frontend est donc impérative.

Une fois que l'utilisateur reçoit l'email et clique sur le lien de confirmation, il est redirigé vers la page de validation du Kernel (voir Figure ??) confirmant que l'adresse e-mail a bien été validée.

Après cette validation, l'utilisateur peut se connecter normalement à la plateforme avec ses identifiants. Il accède alors à son espace personnel sur le tableau de bord KSM (voir Figure ??), avec son profil utilisateur correctement initialisé.

5. Synthèse

L'implémentation de l'inscription multi-tenant s'appuie sur la complémentarité des trois couches du système KSM :

Couche	Rôle principal	Actions clés
Frontend	Interface utilisateur et orchestration	Formulaire d'inscription, capture du code d'org, appel séquentiel découverte/inscription/connexion.
BFF (Proxy)	Passerelle sécurisée	Proxy des routes <code>/api/kernel</code> , injection des secrets <code>X-Client-Id</code> et <code>X-API-Key</code> .

Couche	Rôle principal	Actions clés
Kernel	Logique d'accès et persistance	Résolution des codes d'organisation, validation des tokens d'inscription, création des comptes et isolation des tenants.

Ce protocole garantit une isolation stricte des organisations tout en offrant une expérience utilisateur fluide et sécurisée.

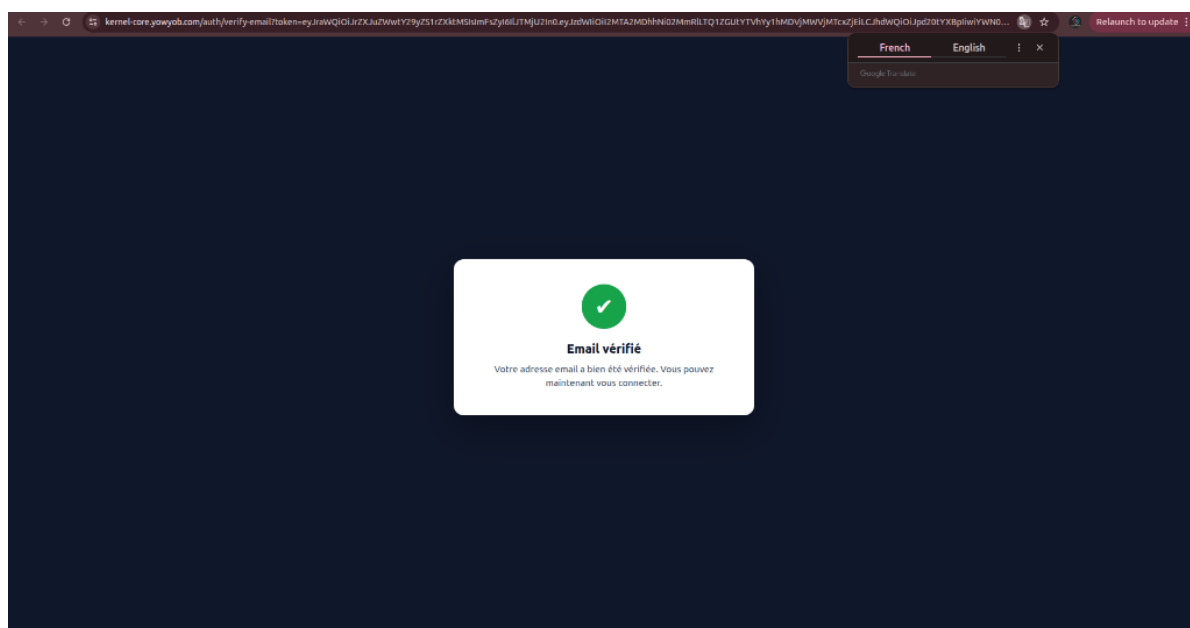


FIGURE 4 – Capture réelle — Page du Kernel de validation de l'e-mail (`verify-email?token=...`) suite au clic sur le lien reçu par e-mail.

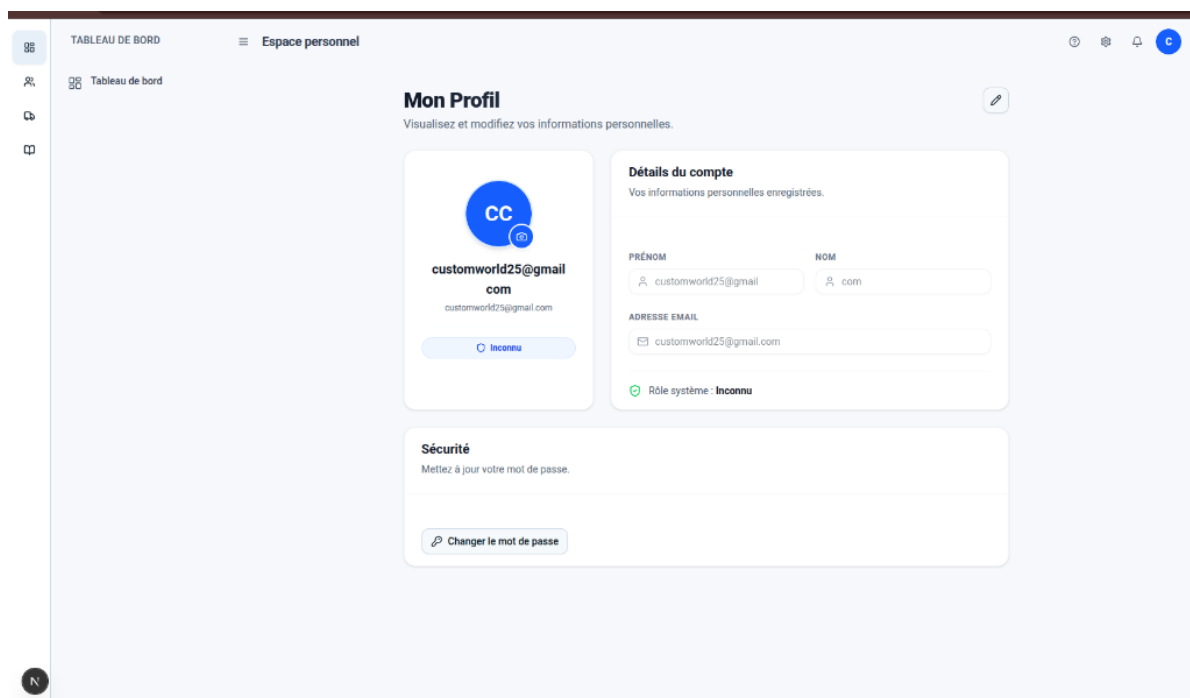


FIGURE 5 – Capture réelle — Tableau de bord de l’espace personnel (Mon Profil) après la première connexion réussie.